

PRD Breakdown: Beginner-Friendly Riverside Alternative

Vision

Build a simplified Riverside.fm-style video podcast app focused on **ease of use for beginners** — clean, straightforward UI, minimal options, reliable host+guest recording with separate tracks, and simple server-side editing.

User Stories & Tasks

1. User Authentication

- **US1:** As a user, I want to create an account with email/password so I can save my podcast sessions.
 - **US2:** As a user, I want to log in securely to access my saved sessions.
 - **Task:** Build signup and login UI with validation.
 - **Task:** Implement backend auth with PostgreSQL user table.
-

2. Session Management

- **US3:** As a host, I want to create a new recording session and get a guest link to share.
 - **US4:** As a guest, I want to join the session without needing an account by clicking a link.
 - **US5:** As a user, I want the app to clearly show the session state (idle, recording, uploading, processing, ready, failed).
 - **Task:** Backend session API with state machine and database schema.
 - **Task:** Frontend UI for session creation, join page, and state display.
-

3. Live Connection & Recording

- **US6:** As a host or guest, I want to see and hear the other person in real-time during the call.

- **US7:** As a host, I want to control recording start and stop, and see visual recording indicators.
 - **US8:** As a participant, I want the browser to record my audio and video locally during the session.
 - **Task:** Implement WebRTC P2P connection with signaling and TURN server.
 - **Task:** Build local recording logic per participant.
 - **Task:** Show recording status and participants' video feeds.
-

4. Uploading

- **US9:** As a user, I want my recorded audio/video to upload automatically after recording stops.
 - **US10:** As a user, I want uploads to be resumable if interrupted.
 - **US11:** As a host, I want to see upload progress for all participants.
 - **Task:** Implement chunked, resumable uploads with progress tracking.
 - **Task:** Backend API to receive and assemble uploads.
-

5. Server-Side Processing & Editing

- **US12:** As a user, I want to trim the start and end of my podcast before exporting.
 - **US13:** As a user, I want the backend to merge my and my guest's tracks into a single video and audio file.
 - **US14:** As a user, I want to see processing progress and be notified when my episode is ready.
 - **Task:** Build backend FFmpeg pipeline for syncing, trimming, and exporting.
 - **Task:** Create editing UI to set trim points and submit instructions.
 - **Task:** Implement job queue with retries and status updates.
-

6. Episode Management

- **US15:** As a user, I want to see a list of my recorded episodes.
 - **US16:** As a user, I want to download final MP4 and MP3 files.
 - **US17:** As a user, I want to delete episodes I no longer need.
 - **Task:** Build episodes list UI.
 - **Task:** Implement download and delete API endpoints.
-

7. Failure Handling

- **US18:** As a user, I want to be warned if a participant disconnects during recording but continue recording without them.
 - **US19:** As a user, I want upload failures to allow resuming or retrying.
 - **US20:** As a user, I want clear error messages if processing fails, with retry options.
 - **Task:** Build UI for warnings and error messages.
 - **Task:** Backend logic for marking partial recordings, retries, and error states.
-

Design Principles (to include in Cursor notes)

- Minimalist UI with no clutter
 - Clear, jargon-free status messages
 - One-click guest join, no account needed
 - Host-only recording controls
 - Simple linear workflow: create → record → upload → edit → export
 - Mobile and desktop browser support (Chrome, Edge, Safari)
-

Summary for Cursor

You're building a **streamlined, beginner-first** video podcast tool, with these pillars:

- Host+guest browser P2P video calls
- Separate local recording + upload per participant

- Basic server-side trimming and merging
- Straightforward session & failure handling
- Clean, no-fluff UI for easy navigation